

STANDARD OPERATING PROCEDURE (SOP)

CRUD-BASED WEB APPLICATION DEVELOPMENT

EMPLOYEE MANAGEMENT SYSTEM

Submitted as part of the
Standard Operating Procedure (SOP) Activity

Project Type : CRUD-Based Web Application

Technology : HTML, CSS, JavaScript, Django REST Framework, SQLite

Name : Preetha M S
Department : Information Technology

1. INTRODUCTION

The Employee Management System is a web-based CRUD application developed to manage employee information in a simple and organized way.

The application allows users to add, view, update, delete, and search employee records. It provides a user-friendly web interface and a REST API for managing employee data.

The frontend is developed using HTML, CSS, and JavaScript. The backend is developed using Python and Django REST Framework, while SQLite is used to store employee information.

This project demonstrates the complete development of a CRUD-based web application, including database management, frontend-backend integration, validation, API testing, and version control using Git and GitHub.

2. PROBLEM STATEMENT

Managing employee information manually can be time-consuming and may lead to errors, duplicate records, or difficulty in maintaining updated information.

A simple digital system is needed to store employee details and perform common operations efficiently.

The Employee Management System solves this problem by providing a web-based application where employee records can be added, viewed, updated, deleted, and searched easily. The system also uses a REST API and database to ensure organized data management.

3. OBJECTIVES

The main objectives of the Employee Management System are:

- To develop a simple and user-friendly Employee Management System.
- To implement Create, Read, Update, and Delete (CRUD) operations.
- To store employee information securely in a database.
- To provide REST API functionality for employee records.
- To provide search functionality for finding employees by name.
- To validate employee information before storing it.
- To connect the frontend with the backend API.
- To test the application using Postman.
- To use Git and GitHub for version control and project management.

4. TECHNOLOGIES USED

The following technologies and tools were used to develop the Employee Management System:

HTML – Used to create the structure of the web page.

CSS – Used to provide styling and improve the appearance of the application.

JavaScript – Used to implement frontend functionality and communicate with the REST API.

Python – Used as the backend programming language.

Django – Used as the web framework for developing the application.

Django REST Framework – Used to create and manage REST API endpoints.

SQLite – Used as the database for storing employee records.

Postman – Used to test the REST API independently.

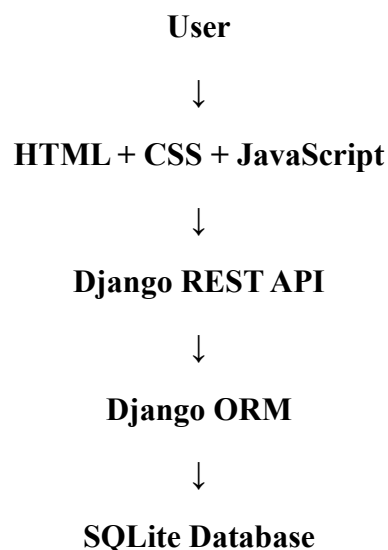
Git – Used for version control and tracking project changes.

GitHub – Used to store and manage the project source code.

Visual Studio Code – Used as the development environment.

5. SYSTEM ARCHITECTURE

The Employee Management System follows a simple three-layer architecture consisting of the frontend, backend, and database.



The user interacts with the application through the web interface. HTML and CSS provide the structure and design, while JavaScript handles user actions and sends requests to the backend.

Django REST Framework processes the API requests and performs the required CRUD operations. Django ORM is used to communicate with the SQLite database, where employee records are stored.

6. DATABASE DESIGN

The Employee Management System uses SQLite as the database. Django ORM is used to create and manage the database table.

The Employee table contains the following fields:

ID – Unique identifier for each employee and the primary key.

Name – Stores the employee's name.

Email – Stores the employee's email address and is configured as unique.

Phone – Stores the employee's phone number.

Department – Stores the department in which the employee works.

Position – Stores the employee's job position.

Salary – Stores the employee's salary.

The database design ensures that employee information is stored in an organized manner. The primary key uniquely identifies each employee, while the unique constraint on the email field helps prevent duplicate email records.

7. CRUD OPERATIONS

CRUD stands for Create, Read, Update, and Delete. These four operations are the main functions of the Employee Management System.

7.1 CREATE

The Create operation is used to add a new employee to the system. The user enters the employee details such as name, email, phone, department, position, and salary, and clicks the Save Employee button. The data is sent to the backend using a POST request and stored in the SQLite database.

7.2 READ

The Read operation is used to retrieve employee records from the database. The application sends a GET request to the REST API and displays the available employee records in a table.

7.3 UPDATE

The Update operation is used to modify existing employee information. When the user clicks the Edit button, the selected employee details are loaded into the form. After making changes, the updated data is sent to the backend using a PUT request.

7.4 DELETE

The Delete operation is used to remove an employee record from the system. When the user clicks the Delete button, a confirmation message is displayed. After confirmation, a DELETE request is sent to the API and the selected employee record is removed from the database.

8. SEARCH FUNCTIONALITY

The Employee Management System provides a search feature to help users find employee records quickly.

A search box is provided above the employee table. When the user enters an employee name, JavaScript filters the available employee records and displays only the matching records.

This feature makes it easier to locate a particular employee when multiple records are present in the system.

9. REST API

The backend provides REST API endpoints for managing employee records. Django REST Framework is used to create the API and handle HTTP requests.

The following endpoints are implemented:

GET /api/employees/

Used to retrieve all employee records.

POST /api/employees/

Used to create a new employee record.

GET /api/employees/{id}/

Used to retrieve the details of a specific employee.

PUT /api/employees/{id}/

Used to update an existing employee record.

DELETE /api/employees/{id}/

Used to delete an employee record.

The REST API acts as a communication layer between the frontend and the database. JavaScript sends requests to the API, and the API returns the required response.

10. VALIDATION

Input validation is used to ensure that employee information is entered correctly before it is stored in the database.

10.1 FRONTEND VALIDATION

The frontend form includes the following validations:

- Name is required.
- Email is required.
- Phone number is required.
- Department is required.
- Position is required.
- Salary is required.
- The email field checks for a valid email format.

These validations help prevent incomplete or incorrectly formatted data from being submitted.

10.2 BACKEND VALIDATION

Django REST Framework validates the employee data received through the API before saving it to the database.

The email field is configured as unique, which prevents duplicate email addresses from being stored for different employees.

11. TESTING PROCEDURE

The Employee Management System was tested through both the web interface and Postman to verify that the application and REST API work correctly.

11.1 FRONTEND TESTING

The following operations were tested:

- Adding a new employee.
- Viewing employee records.
- Updating employee details.
- Deleting an employee.
- Searching for an employee by name.
- Checking required field validation.
- Checking invalid email format.

The frontend tests confirmed that the main employee management functions work correctly.

11.2 API TESTING USING POSTMAN

Postman was used to test each REST API endpoint independently.

The following API operations were tested:

- GET request – Used to retrieve employee records. Response: 200 OK.
- POST request – Used to create a new employee. Response: 201 Created.

- PUT request – Used to update an employee. Response: 200 OK.
- DELETE request – Used to delete an employee. Response: 204 No Content.

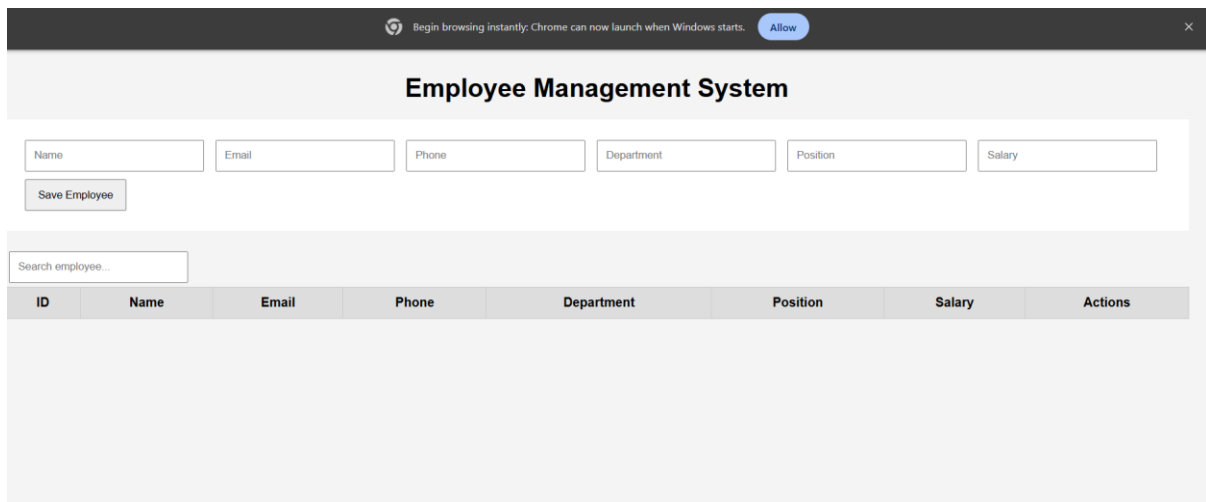
The response status codes were checked after each request to verify successful API execution.

11.3 TESTING RESULTS

The main CRUD operations were successfully tested. The frontend was able to communicate with the backend API, and employee records were stored and managed using the SQLite database.

12. SCREENSHOTS AND TEST RESULTS

12.1 Main Application : Employee Management System Home Page



The screenshot shows a web browser window with a dark header bar containing a notification: "Begin browsing instantly: Chrome can now launch when Windows starts." with an "Allow" button. The main content area has a light gray background and is titled "Employee Management System" in bold black text. Below the title, there is a form with six input fields: "Name", "Email", "Phone", "Department", "Position", and "Salary". A "Save Employee" button is located below the "Name" field. Below the form, there is a search bar with the placeholder text "Search employee...". At the bottom, there is a table with the following headers: "ID", "Name", "Email", "Phone", "Department", "Position", "Salary", and "Actions". The table body is currently empty.

ID	Name	Email	Phone	Department	Position	Salary	Actions
----	------	-------	-------	------------	----------	--------	---------

Employee Management System

Save Employee

ID	Name	Email	Phone	Department	Position	Salary	Actions
2	Arun	arun0712@gmail.com	3456541233	IT	Software Development	40000.00	Edit Delete
3	Priya	priya0987@gmail.com	3456541233	IT	Software Developer	50000.00	Edit Delete

12.2 Adding an Employee : Adding a New Employee

Employee Management System

Save Employee

ID	Name	Email	Phone	Department	Position	Salary	Actions
2	Arun	arun0712@gmail.com	3456541233	IT	Software Development	40000.00	Edit Delete
3	Priya	priya0987@gmail.com	3456541233	IT	Software Developer	50000.00	Edit Delete

12.3 Updated Employee Record : Updating Employee Details

Employee Management System

ID	Name	Email	Phone	Department	Position	Salary	Actions
2	Arun	arun0712@gmail.com	3456541233	IT	Software Development	45000.00	Edit Delete
3	Priya	priya0987@gmail.com	3456541233	IT	Software Developer	50000.00	Edit Delete
4	Periyamayaki	peri26@gmail.com	9487736278	IT	Tester	55000.00	Edit Delete

12.4 Search Functionality : Searching Employee by Name

Employee Management System

ID	Name	Email	Phone	Department	Position	Salary	Actions
4	Periyamayaki	peri26@gmail.com	9487736278	IT	Tester	55000.00	Edit Delete

12.5 Delete Operation : Deleting an Employee Record

Employee Management System

Name

Email

Phone

Department

Position

Salary

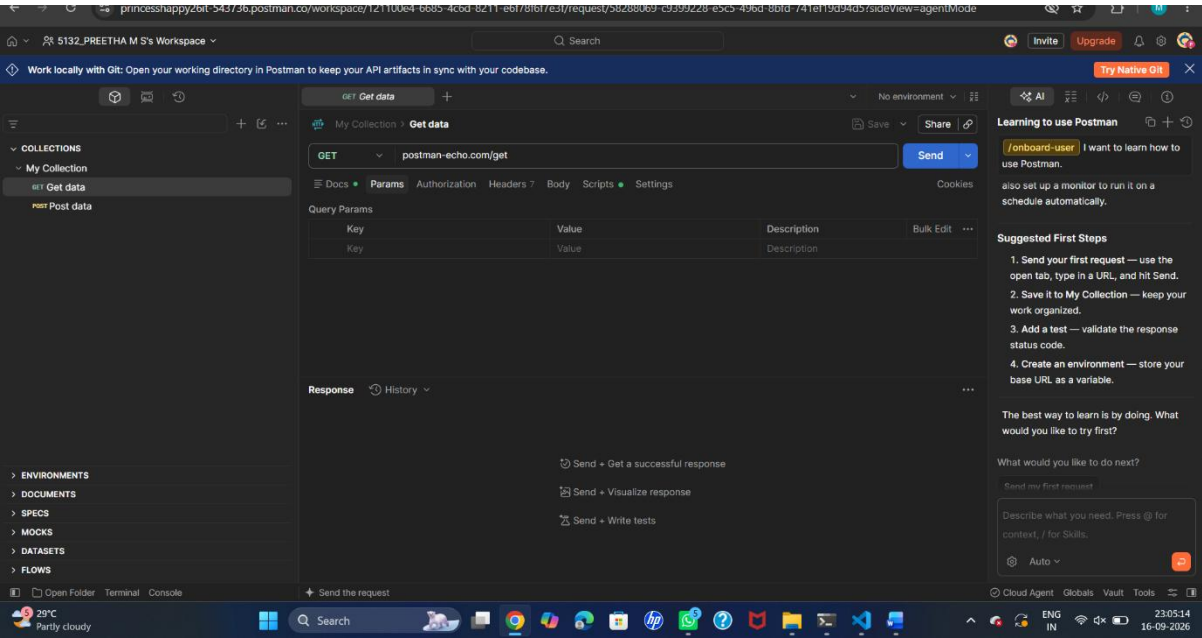
Save Employee

Search employee...

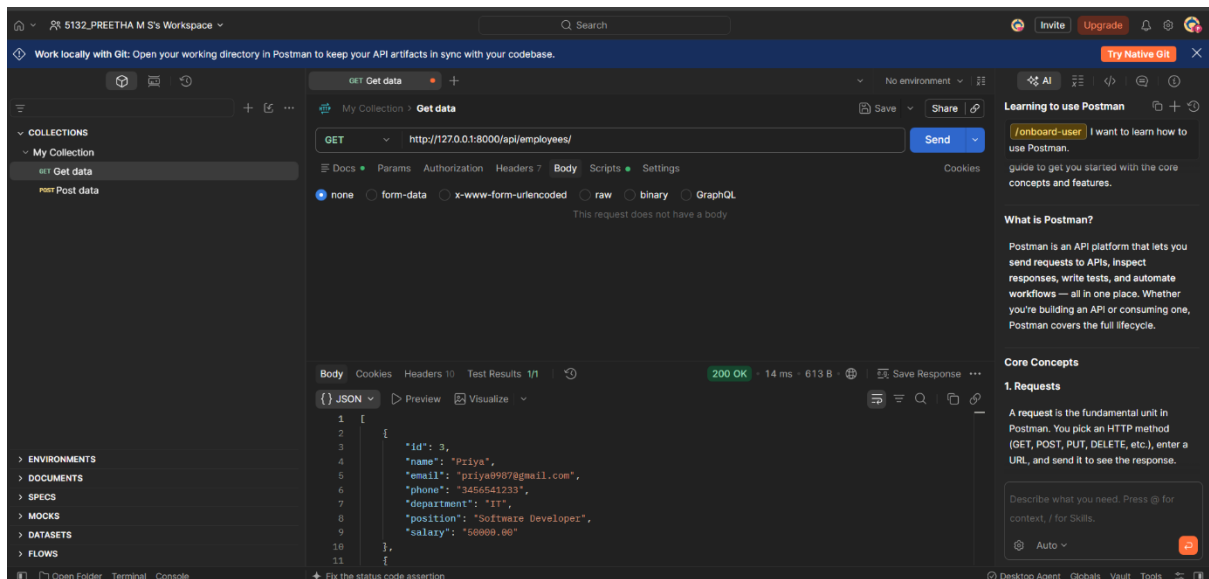
ID	Name	Email	Phone	Department	Position	Salary	Actions
3	Priya	priya0987@gmail.com	3456541233	IT	Software Developer	50000.00	Edit Delete
4	Periyanayaki	peri26@gmail.com	9487736278	IT	Tester	55000.00	Edit Delete

API screenshots :

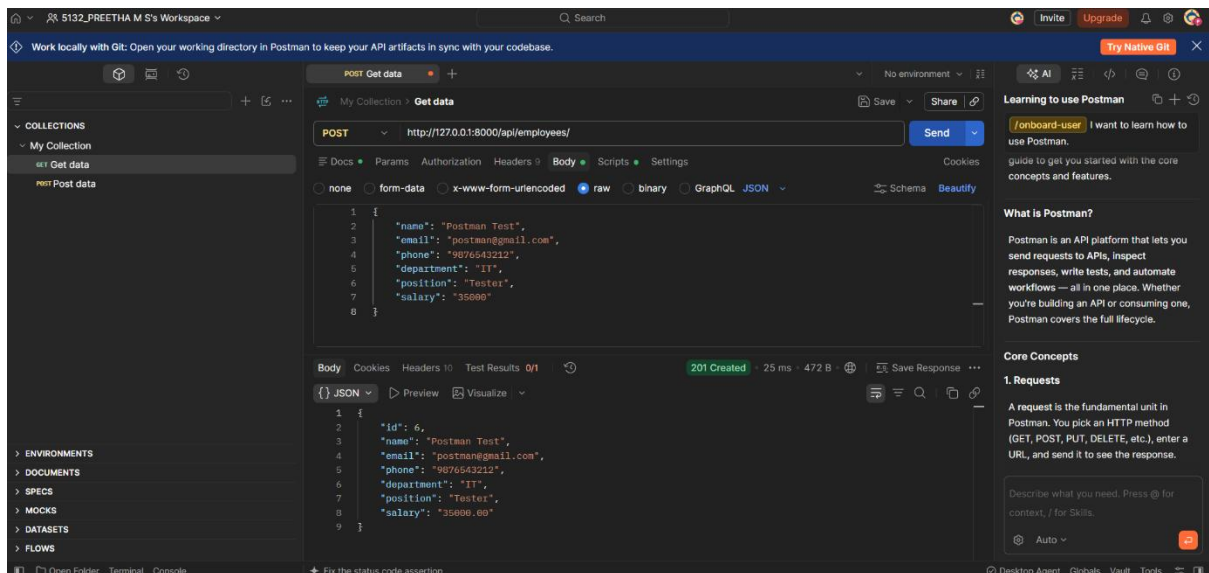
12.6 REST API



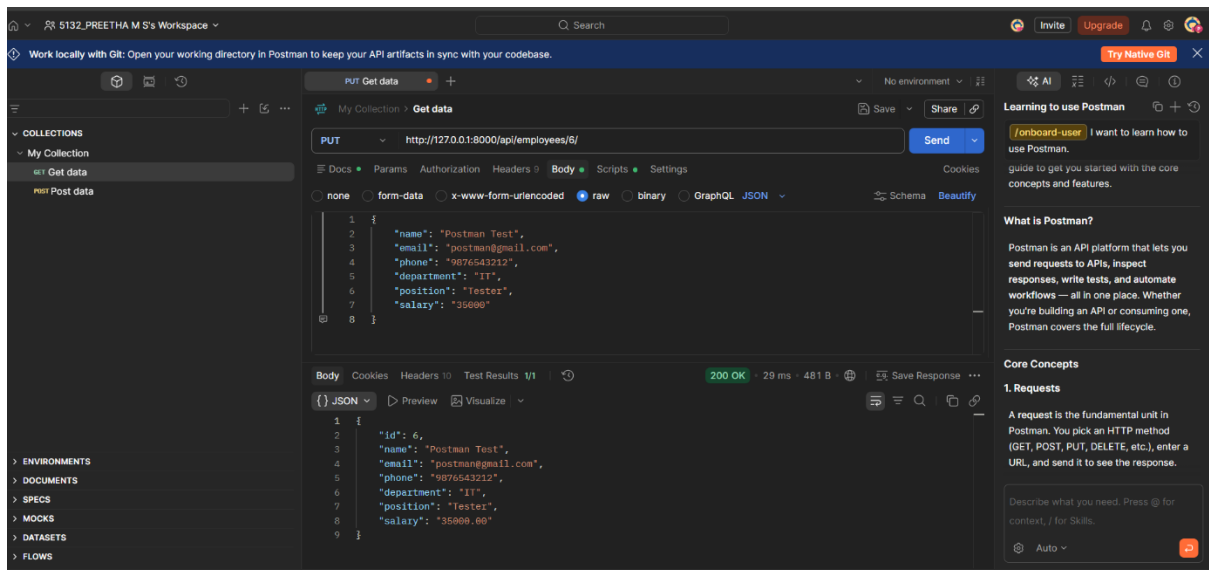
12.7 GET Request : GET API Testing – 200 OK



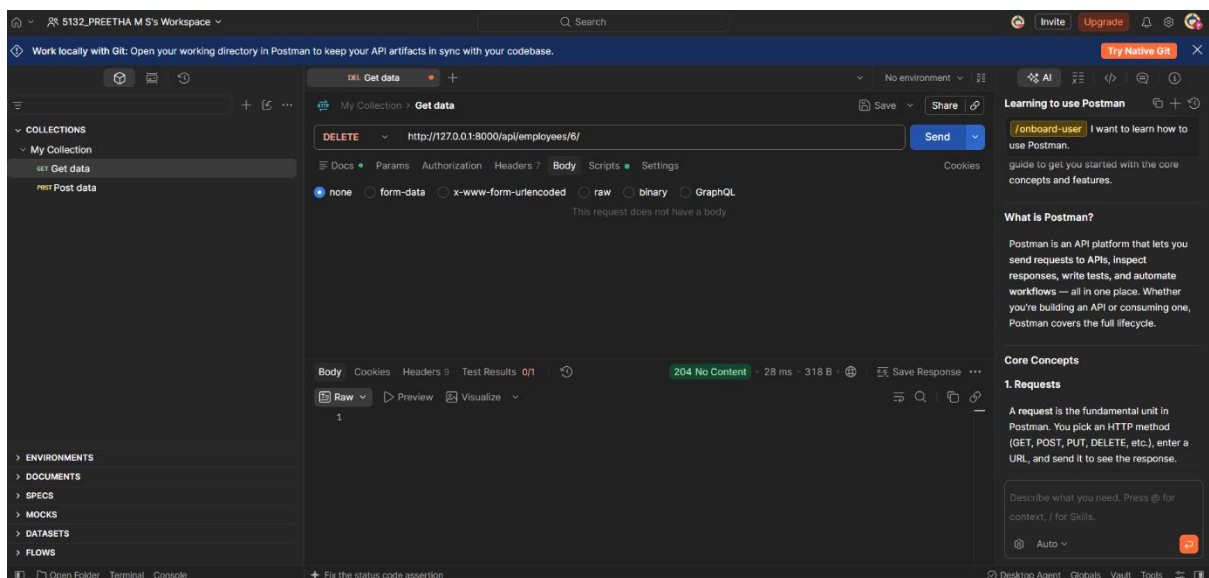
12.8 POST Request : POST API Testing – 201 Created



12.9 PUT Request : PUT API Testing – 200 OK



12.10 DELETE Request : DELETE API Testing – 204 No Content



13. CHALLENGES AND SOLUTIONS

During the development of the Employee Management System, a few challenges were faced and resolved.

13.1 Connecting Frontend and Backend

Challenge:

The frontend needed to communicate with the Django backend.

Solution:

JavaScript Fetch API was used to send HTTP requests to the Django REST API and receive responses.

13.2 Database Management

Challenge:

Employee information needed to be stored and managed properly.

Solution:

Django ORM and SQLite were used to create and manage the Employee database table.

13.3 Input Validation

Challenge:

Empty fields, invalid email formats, and duplicate email addresses could affect data accuracy.

Solution:

Frontend HTML validation and Django REST Framework validation were used to validate the employee data.

13.4 API Testing

Challenge:

The REST API needed to be tested independently from the frontend.

Solution:

Postman was used to test the GET, POST, PUT, and DELETE API requests and verify their response status codes.

14. VERSION CONTROL

Git was used for version control during the development of the Employee Management System.

The project was initialized as a Git repository and changes were committed during development. A .gitignore file was added to prevent unnecessary files such as the virtual environment and Python cache files from being tracked.

The completed project was pushed to GitHub for source code management, backup, and submission.

GitHub Repository :

<https://github.com/PreethaMS-code/EmployeeManagement>

15. FUTURE ENHANCEMENTS

The following features can be added in future versions of the Employee Management System:

- User login and authentication.
- Role-based access control.
- Employee profile management.
- Department-wise filtering.
- Salary reports.
- Employee attendance management.
- Dashboard with employee statistics.
- Export employee records to CSV or PDF.
- Improved mobile responsiveness.

16. CONCLUSION

The Employee Management System successfully demonstrates the development of a CRUD-based web application using HTML, CSS, JavaScript, Django REST Framework, and SQLite.

The system provides a simple interface for managing employee records and supports Create, Read, Update, and Delete operations through both the web interface and REST API.

The project also demonstrates database management, input validation, frontend-backend integration, API testing using Postman, and version control using Git and GitHub.

Overall, the project provides practical experience in developing and testing a complete web application using modern development tools and technologies.

17. GITHUB REPOSITORY

The complete source code of the Employee Management System is available on GitHub.

Repository:

<https://github.com/PreethaMS-code/EmployeeManagement>

18. AUTHOR

Developed as part of the college Standard Operating Procedure (SOP) activity.

Project: Employee Management System

Project Type: CRUD-Based Web Application

Backend: Django REST Framework

Database: SQLite

API Testing: Postman

Version Control: Git and GitHub

